TERRAFORM STATE MANAGEMENT

# The Terraform mistake that can break your infrastructure team

State is Terraform's memory of what exists. Lose it, corrupt it or leak it, and every plan becomes a guess.

### ✗ Risky

- **Local state** on one laptop
- **State committed to Git**, secrets included
- **No locking:** two applies at once
- **One giant state** for everything

### ✓ Safe

- **Remote backend:** S3 / Azure Blob / GCS
- **Locking** on every plan and apply
- **Encrypted, versioned,** least-privilege access
- **Split state** by env and component

## CHANGE STATE SAFELY

**01** Plan in CI, never on a laptop

**02** Lock acquired automatically

**03** Review the plan, then apply

**04** Refactor with moved / import blocks

**Remember:** state can hold secrets in plain text. Treat the state bucket like a password vault.

01 · THE HOOK

# The Terraform mistake that breaks teams is rarely bad HCL

It's state.

Someone runs apply from a laptop with stale local state. Two pipelines apply at the same time. A state file with database passwords gets committed to Git.

Every one of these is preventable, and every one is painful to clean up.

02 · THE PROBLEM

# What state actually is

State is Terraform's record of **which real resources belong to which configuration**: their IDs, attributes and dependencies.

- → **Lost state:** Terraform no longer knows what it manages, and plans try to create everything again
- → **Concurrent applies:** two writers at once risk corrupted state
- → **Leaked state:** state can contain sensitive values **in plain text**
- → **One giant state:** slow plans, and every change risks everything

03 · THE ARCHITECTURE

# A safe remote-state flow

**1 · CI runner (not a laptop)**

Short-lived cloud credentials via OIDC; plan and apply run in the pipeline

**2 · Lock acquired**

A second run waits or fails instead of writing over the first

**3 · Read state → plan → reviewed apply**

The plan output is reviewed before apply, just like a code review

**4 · New state version written**

Encrypted bucket with versioning on, so every previous version is recoverable

**Access: least privilege**

Only the pipeline role writes. Very few humans can read.

# Remote backends in all three clouds

```
# AWS: S3 with native locking (Terraform 1.10+)
backend "s3" {
  bucket       = "acme-tfstate-prod"
  key          = "network/terraform.tfstate"
  region       = "us-east-1"
  encrypt      = true
  use_lockfile = true
}
# Azure: locking via blob leases
backend "azurerm" {
  resource_group_name  = "rg-tfstate"
  storage_account_name = "acmetfstateprod"
  container_name       = "tfstate"
  key                  = "network.tfstate"
}
# GCP: locking supported by the gcs backend
backend "gcs" {
  bucket = "acme-tfstate-prod"
  prefix = "network"
}
```

# Change and recover state safely

```
# Rename without destroy/recreate (TF 1.1+)
moved {
  from = aws_instance.web
  to   = aws_instance.app
}
# Adopt an existing resource (TF 1.5+)
import {
  to = aws_s3_bucket.logs
  id = "acme-logs"
}
```

```
# Detect drift without changing anything
terraform plan -refresh-only
# Last resort, only if the lock is truly stale
terraform force-unlock LOCK_ID
```

> **Recovery:** with bucket versioning on, a bad write is fixed by restoring the previous state version.

# How small should each state be?

**Split state**

✓ Small blast radius
✓ Fast plans
✓ Clear ownership
✗ Dependencies between states need managing

**One big state**

✓ Simple references
✗ Slow plans
✗ Every change risks everything
✗ Broad access to sensitive data

**Watch out:** the terraform_remote_state data source needs read access to another config's whole state. Consider publishing just the outputs you need to a parameter store instead.

06 · THE TAKEAWAY

# Terraform state checklist

- ☐ Remote backend for every environment, no local state
- ☐ State files never committed to Git (.gitignore them)
- ☐ Locking on for every plan and apply
- ☐ Encryption at rest and bucket versioning on
- ☐ Write access only for the pipeline role
- ☐ State split by environment and component
- ☐ Refactors done with moved / import blocks
- ☐ Restore procedure written down and tested

07 · YOUR TURN

# How does your team protect Terraform state from accidental changes?

Share your safeguards in the comments.

**Follow Deji Ajishe**

for the rest of this 30-part multi-cloud DevOps series

♻️ Repost or share this with an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a